ServiceHub Backend: What Is in Place

In one sentence

This is the starter structure for the ServiceHub backend. A scaffold is the foundation of a project: it gives later features a place to go, but it does not mean those features have been built.

What works today

- The server starts with ‘npm run dev‘.
- ‘GET /api/v1/health‘ responds with ‘{ "status": "ok" }‘.
- One automated test checks that health response.
- ‘npm test‘ and ‘npm run lint‘ both pass.
- Environment settings are checked with Zod when the app starts.
- Requests are logged, common security headers are set, CORS and JSON parsing are configured, and API requests are rate limited.
- Errors go through one handler and use a consistent JSON shape: ‘{ "error": { "code": "...", "message": "..." } }‘.

How the folders are organized

- ‘src/app.js‘ assembles Express middleware and routes. ‘src/server.js‘ starts the listening server.
- ‘src/config/‘ validates settings. ‘src/middlewares/‘ contains request handling stubs and middleware.
- ‘src/modules/‘ groups code by feature. The planned modules are auth, users, businesses, services, staff, schedules, appointments, reviews, notifications, dashboard, audit logs, and platform admin. Each has routes, controller, service, repository, and validation placeholder files.
- ‘src/database/‘ contains the Prisma client setup and a transaction helper. ‘prisma/schema.prisma‘ defines PostgreSQL as the database provider but has no business models yet.
- ‘src/lib/‘, ‘src/jobs/‘, and ‘src/realtime/‘ hold placeholders for logging, Redis/BullMQ jobs, and Socket.IO.
- ‘tests/‘ contains the health endpoint test. ‘docs/‘ describes architecture, API conventions, and tenancy guidance.

What is not built yet

Authentication, business and user features, appointment flows, database models, real Redis connections, job workers, and Socket.IO events are not implemented. The auth, tenant-context, RBAC, and validation middleware files currently pass requests through; they do not secure routes yet. The health endpoint is the only implemented route.

Tenant design

The documented plan is one shared PostgreSQL database. Future tenant-owned tables should include ‘businessId‘; tenant repositories should include that ID in every relevant query. Tenant context should come from an authenticated business membership. This is design guidance only until the models and middleware are implemented.

Run it locally

Requires Node.js 22 or newer. From the project folder:

1. Run ‘npm install‘.
2. Optionally copy ‘.env.example‘ to ‘.env‘ and adjust local settings.
3. Run ‘npm run dev‘.
4. Open ‘http://localhost:3000/api/v1/health‘.
5. Run ‘npm test‘ for the health test or ‘npm run lint‘ for linting.

The default database and Redis URLs are configuration values only; the app does not connect to those services yet.

Suggested build order

1. Authentication and user identity.
2. Tenant context and business-scoped repository filters.
3. Services and staff.
4. Schedules.
5. Appointments.

Git status

The scaffold was pushed to the ‘develop‘ branch in commit ‘4f2aef8‘ (‘Scaffold ServiceHub backend‘). The blueprint PDF in the project folder was left uncommitted.